

Protocol Audit Report

Rsymp

May 30, 2026



Protocol Audit Report

Version 1.0

Rsy.io

May 30, 2026

Protocol Audit Report

Rsynd

May 30, 2026

Prepared by: RSYND Lead Auditors: - Rsynd - Patrick

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Storing the password onchain makes it visible to everyone , and no longer private
 - * [H-2] `PasswordStore::setPassword` has no access control ,means a non owner can change the password
 - Informational
 - * [I-1] The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect.

Protocol Summary

PasswordStore is a protocol designed to store and retrieve the password of a user. Only the owner can set and be able to access this password

Disclaimer

The RSYND team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of

the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
Likelihood	High	High	Medium	Low
	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Commit Hash:

2e8f81e263b3a9d18fab4fb5c46805ffc10a9990

Scope

- In Scope:

`./src/`

`#-- PasswordStore.sol`

- Solc Version: 0.8.18
- Chain(s) to deploy contract to: Ethereum

Roles

- Owner: Is the only one who should be able to set and access the password.
- For this contract, only the owner should be able to interact with the contract.

Executive Summary

Issues found

Severity	Number of issues found
High	2
Medium	0

then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the stored password. However, you're also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with this decryption key.

[H-2] PasswordStore::setPassword has no access control ,means a non owner can change the password

Description: The PasswordStore::setPassword function is set to be an external function, however the purpose of the smart contract and function's natspec indicate that This function allows only the owner to set a new password.

```
function setPassword(string memory newPassword) external {
@>    // @Audit - There are no Access Controls.
    s_password = newPassword;
    emit SetNewPassword();
}
```

Impact: Anyone can change/set the password ,severely breaking the contract intended functionality

Proof of Concept: Add the following to the PasswordStore.t.sol

```
function test_anyone_can_set_password(address randomAddress) public {
    vm.assume(randomAddress != owner);
    vm.prank(randomAddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();

    assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation: Add an access control conditional to PasswordStore::setPassword.

```
if(msg.sender!= s_owner){
revert PasswordStore__NotOwner();
}
```

Informational

[I-1] The PasswordStore::getPassword natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect.

Description:

```
/*
 * @notice This allows only the owner to retrieve the password.
@>  * @param newPassword The new password to set.
 */
function getPassword() external view returns (string memory)
```

PassswordStore::getPassword function signature is `getPassword()` while the natspec says it should be `getPassword(string)`.

Impact: The natspec is incorrect

Recommended Mitigation: Remove the incorrect natspec line.

```
-      * @param newPassword The new password to set.
```